

Maschinelles Lernen

9. Autoencoder

Prof. Dr. Dennis Janka

Hochschule Karlsruhe
University of
Applied Sciences



Motivation

Unüberwachtes Lernen:

- Dimensionsreduktion: Niedrig-dimensionale Repräsentation eines hochdimensionalen Featureraumes finden, z.B. **Featurelearning** für nachfolgendes Überwachtes Lernen
- Matrixfaktorisierung: z.B. für Empfehlungsdienste
—> Vorlesungsteil Prof. Preisach, Kapitel Transformationen
- Anomaliedetektion: z.B. für Prozessindustrie, Kreditkartenzahlungen, ...

Inhalt

9.1 Einführung

9.2 Lineare Autoencoder und PCA

9.3 Vortrainieren mit Autoencodern

9.4 Convolutional Autoencoder

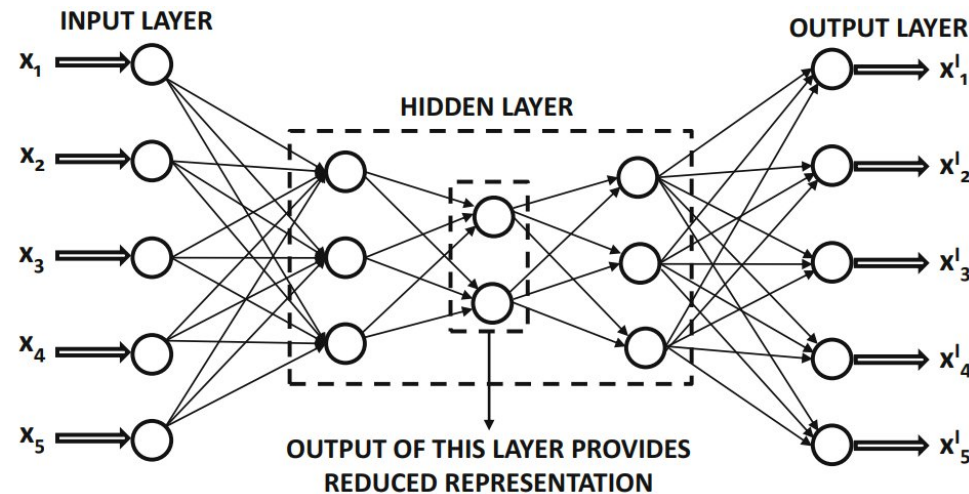
9.5 Anwendung: Anomalie-Erkennung mit Autoencodern

9.1 Einführung

Was ist ein Autoencoder?

Ein Autoencoder ist ein bestimmter Typ von Architektur, bei dem die Ausgabeschicht dieselbe Dimension wie die Eingabeschicht hat

Beim Training wird versucht, jede Dimension so gut wie möglich zu rekonstruieren, also $x_i \approx x'_i$



Aber: Anzahl der Neuronen in der mittleren Schicht ist typischerweise geringer als Input/Output-Dimension

—> Neuronen in der mittleren Schicht enthalten eine reduzierte Darstellung des Inputs!
(Autoencoder machen also eine Form der verlustbehafteten Kompression)

Autoencoder: Aufbau

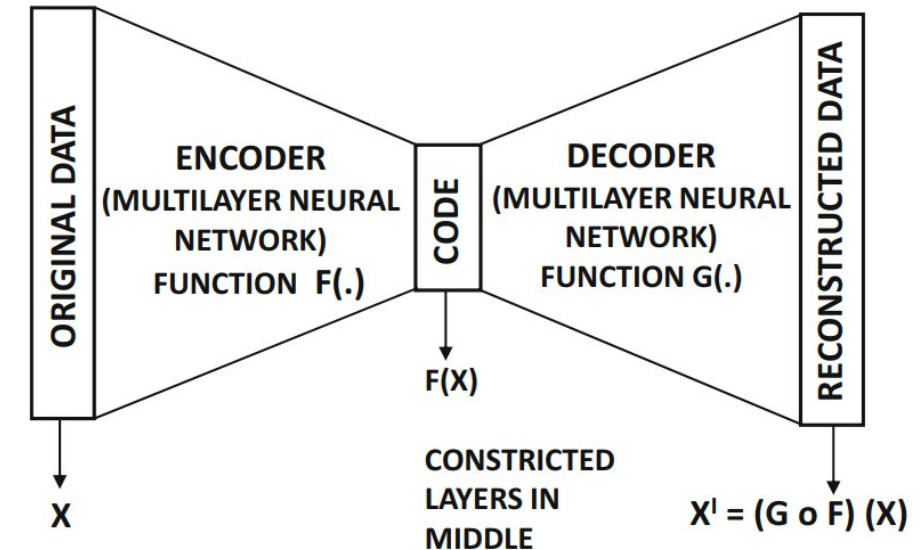
- Mindestens 3 Schichten: Eingabe, Code, Ausgabe
- Meist ungerade Anzahl Schichten M
- Oft symmetrische Architektur (Anzahl Neuronen / Verbindungen in k -ter Schicht = Anzahl in $M-k+1$ -ter Schicht)
- Mittlere Schicht hat meist die wenigsten Neuronen („Bottleneck“). Bezeichnung: *Code* oder *latent space*.

Training: MSE: Abstand zwischen Original und Rekonstruktion

Summe über alle Trainingsbeispiele, über jeden Eintrag.

$$\min_w \frac{1}{N} \sum_{i,j} (x_{ij} - x'_{ij})^2 (+ \text{Regularisierung})$$

Nach dem Training: Output der mittleren Schicht enthält den Code.

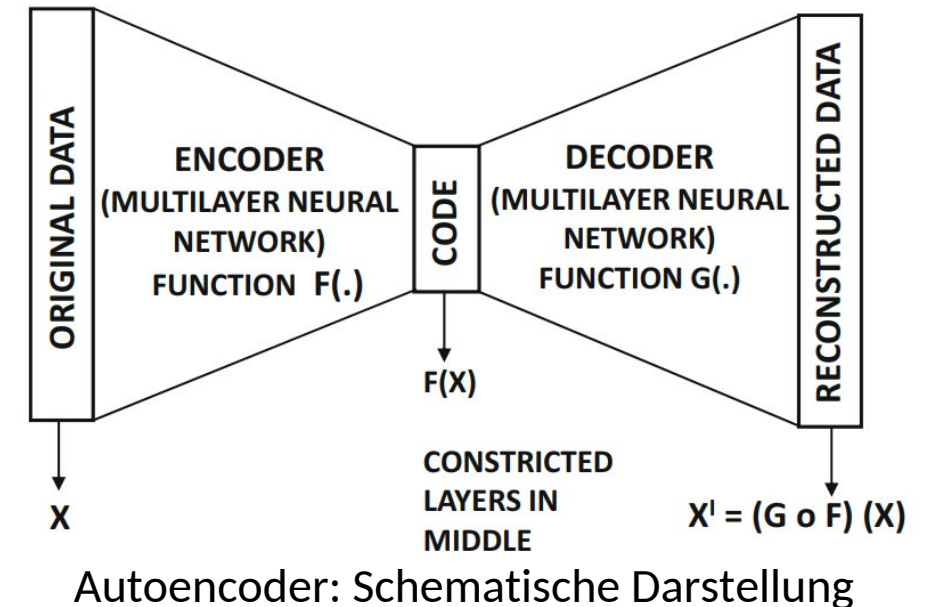


Autoencoder: Aufbau

Allgemein steht für Encoder und Decoder gesamtes Spektrum des NN-Baukastens zur Verfügung:

- Beliebig viele Schichten (Deep Autoencoder)
- Faltungsschichten (Convolutional Autoencoder)
- Verschiedene Aktivierungsfunktionen
- Verschiedene Regularisierungstechniken
(damit kann sogar die mittlere Schicht größer sein als Input/Output Schicht – Regularisierung gewährleistet, dass sie dünnbesetzt ist → sparse Autoencoder)
- Achtung: Operationen sind i.d.R. nicht umkehrbar (z.B. Max-Pooling), d.h. völlige Symmetrie wird normalerweise nicht erreicht

Zunächst einfachster Fall: 1 verdeckte Schicht, keine nichtlineare Aktivierungsfunktion.



9.2 Lineare Autoencoder und PCA

Autoencoder: Linearer Fall

Vorhersage für ein Beispiel (Spaltenvektor -> Spaltenvektor):

$$x' = VUx$$

Vorhersage für ein Beispiel (Zeilenvektor -> Zeilenvektor):

$$x'^T = (VUx)^T = x^T U^T V^T$$

Damit kann man die Vorhersage für die gesamte Featurematrix X (=alle Trainingsbeispiele) schreiben als:

$$X' = XU^T V^T$$

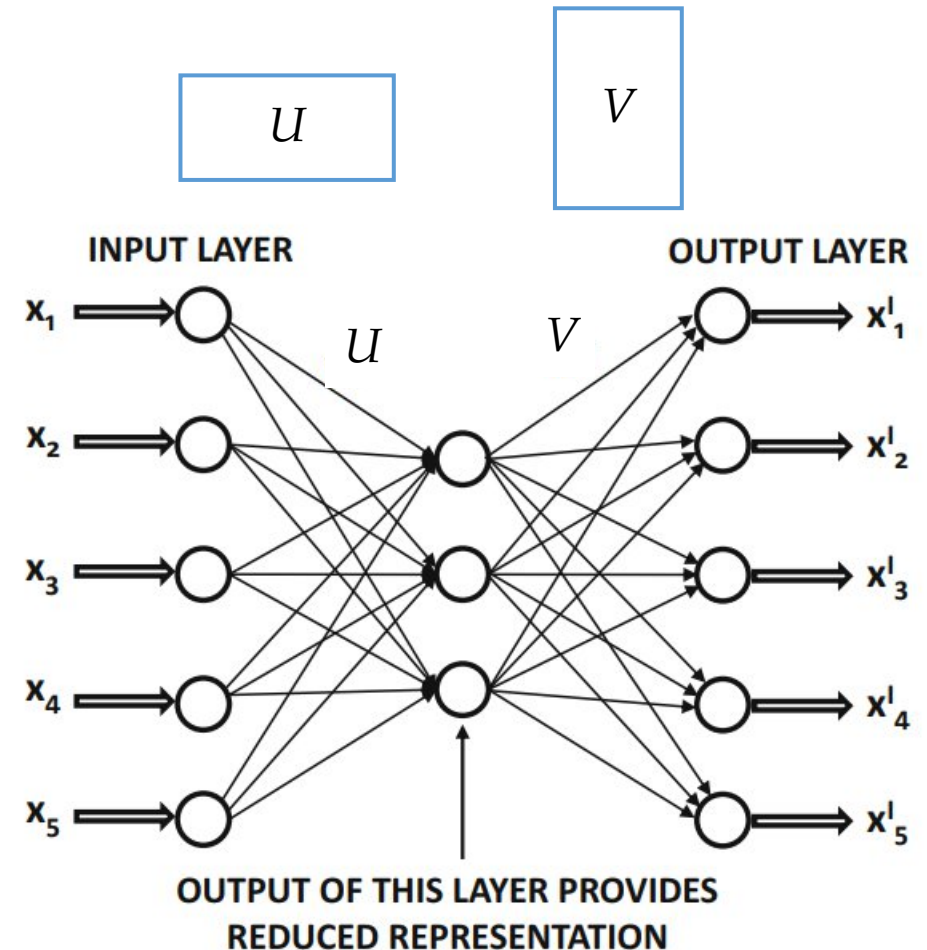
Im Training versucht man also folgende Verlustfunktion zu minimieren:

$$\min_{U,V} \|X - X'\|_F^2 = \min_{U,V} \|X - XU^T V^T\|_F^2$$

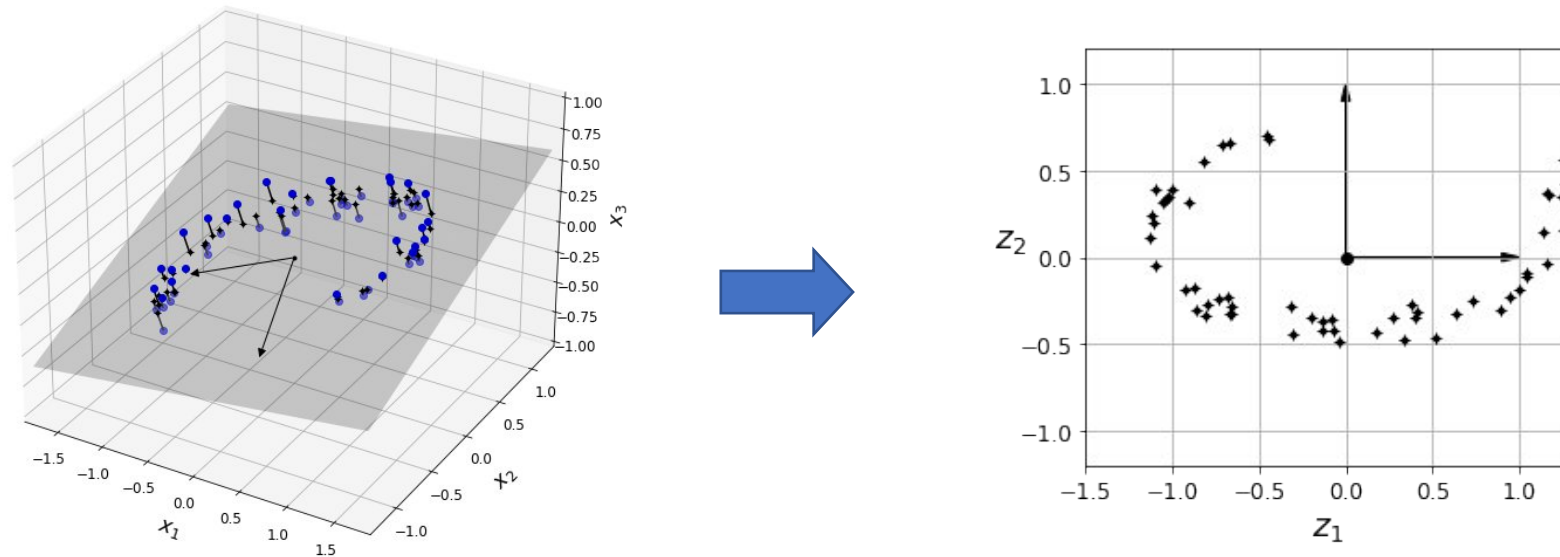
Quadrierte Frobenius Norm

$\|X - XU^T V^T\|_F^2$, d.h. die Summe der quadrierten Einträge der Matrix.

(Das ist einfach die least-squares Verlustfunktion für Regressionsprobleme!)



Erinnerung: PCA



Dimensionsreduktion durch Projektion auf linearen Unterraum, so dass der Rekonstruktionsfehler minimal wird (alternativ: dass die Varianz im Unterraum maximal wird).

Anleitung:

- Für die standardisierten Trainingsdaten berechne Eigenwertzerlegung von $X^T X = U \Sigma U^T$.
- Die ersten k Zeilen von U , $U_k \in \mathbb{R}^{k \times p}$ sind die Projektionsmatrix, mit der man Featurevektoren $x \in \mathbb{R}^p$ auf den k -dimensionalen reduzierten Merkmalsraum abbildet, also

$$z = U_k x \in \mathbb{R}^k$$

Eigenschaften der PCA-Projektionsmatrix U_k

- Man kann die niedrigdimensionalen Vektoren $z = U_k x$ wieder in den ursprünglichen Raum abbilden, indem wir die Abbildung U_k „rückgängig“ machen (so weit es eben geht, durch die Projektion verliert man ja Information).
- Da die Zeilen von U_k orthogonal sind, geschieht dies durch die Transponierte:
$$x' = U_k^T z = U_k^T U_k x$$
- Man kann zeigen: Die Matrix U_k , die die Eigenvektoren zu den k größten Eigenwerten von $X^T X$ enthält liefert die Rekonstruktion mit dem geringsten Fehler, d.h. sie minimiert $\|x - x'\|_2^2$, summiert über alle Trainingsdatenpunkte:

$$\sum_{i=1}^N \|x_i - x'_i\|_2^2 = \sum_{i=1}^N \|x_i - U_k^T U_k x_i\|_2^2$$

- In Matrixschreibweise: U_k ist die Matrix der Dimension $k \times p$, die folgenden Ausdruck minimiert:

$$\|X - XU_k^T U_k\|_F^2$$

Lineare Autoencoder und PCA

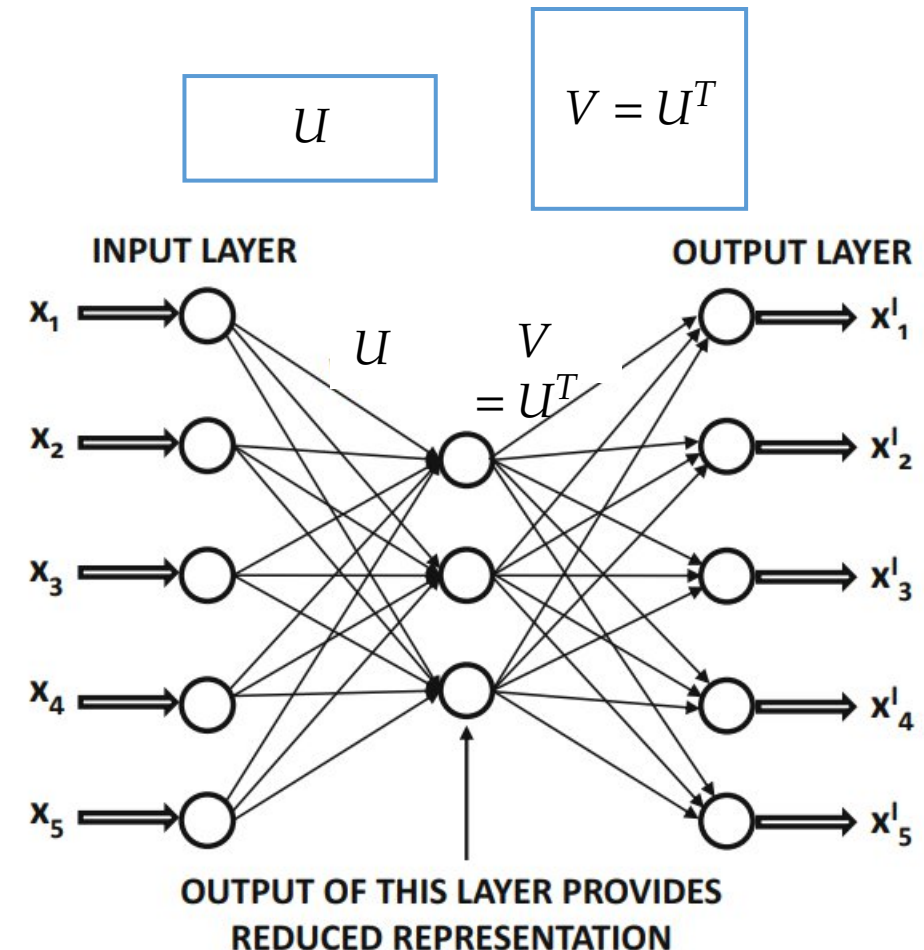
Die Verlustfunktion beim Training von linearen Autoencodern

$$\|X - XU^T V^T\|_F^2$$

hat verschiedene Minima, u.a. die Matrix U_k aus der PCA.

Die Zeilen von U sind aber nicht notwendigerweise orthonormal wie bei der PCA, allerdings ist der aufgespannte Unterraum ist der gleiche.

Wenn man allerdings $V = U^T$ setzt (die Anzahl der Gewichte im Netz also halbiert), „simuliert“ man damit eine PCA!

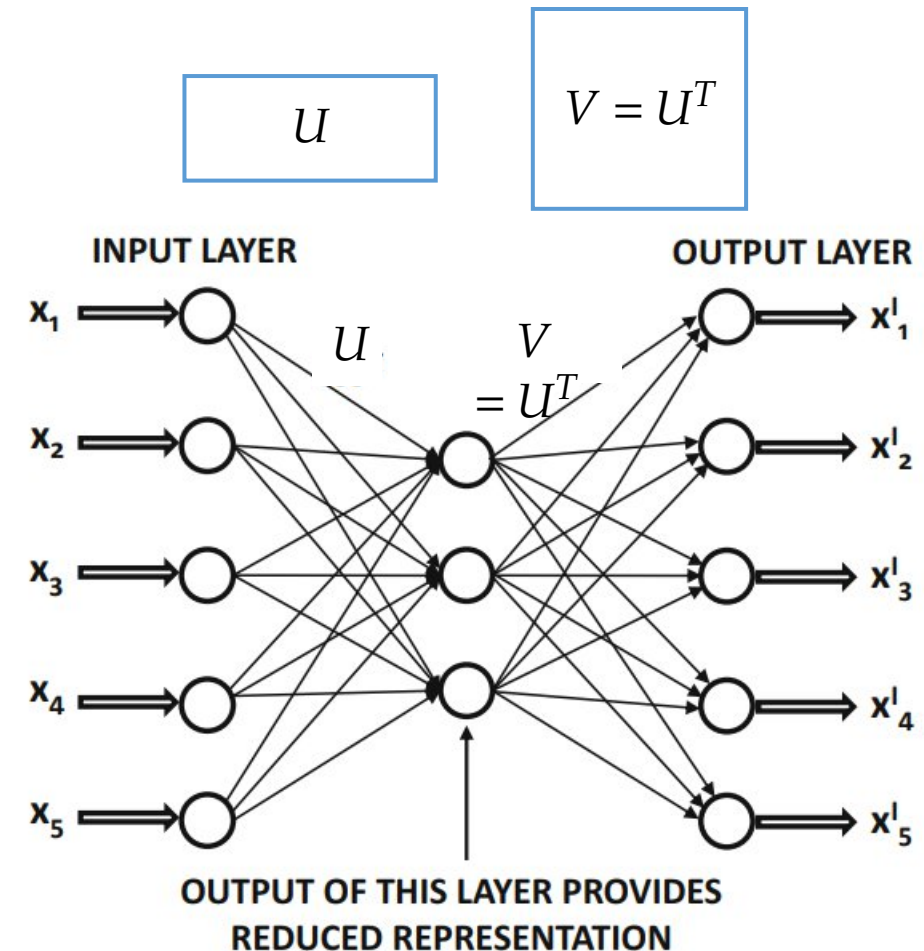


Lineare Autoencoder und PCA

Zusammenfassend:

Lineare Autoencoder als PCA:

- Trainiere linearen Autoencoder mit den gleichen Gewichten für Encoder und Decoder
- Entferne Ausgabeschicht des trainierten Netzes, so dass die verdeckte Schicht die Ausgabeschicht ist
- Das Netz kann nun auf beliebige Datenpunkte angewendet werden und führt damit eine PCA auf die k Hauptkomponenten durch



Spektrale Ansätze vs. Autoencoder

Mit linearen Autoencodern kann man spektrale, d.h. auf Eigenwerten basierende Ansätze wie SVD, PCA, Matrixfaktorisierung in der Sprache der neuronalen Netze ausdrücken.

Diese sind theoretisch gut analysiert und es existieren sehr effiziente numerische Verfahren (d.h. in der Praxis würde man eine SVD eher nicht per Autoencoder berechnen).

Aber: Mit wenigen Modifikationen kann man Autoencoder modifizieren, ohne dass man am Berechnungsframework (NN + SGD + Backpropagation) etwas ändern müsste.

Beispiel:

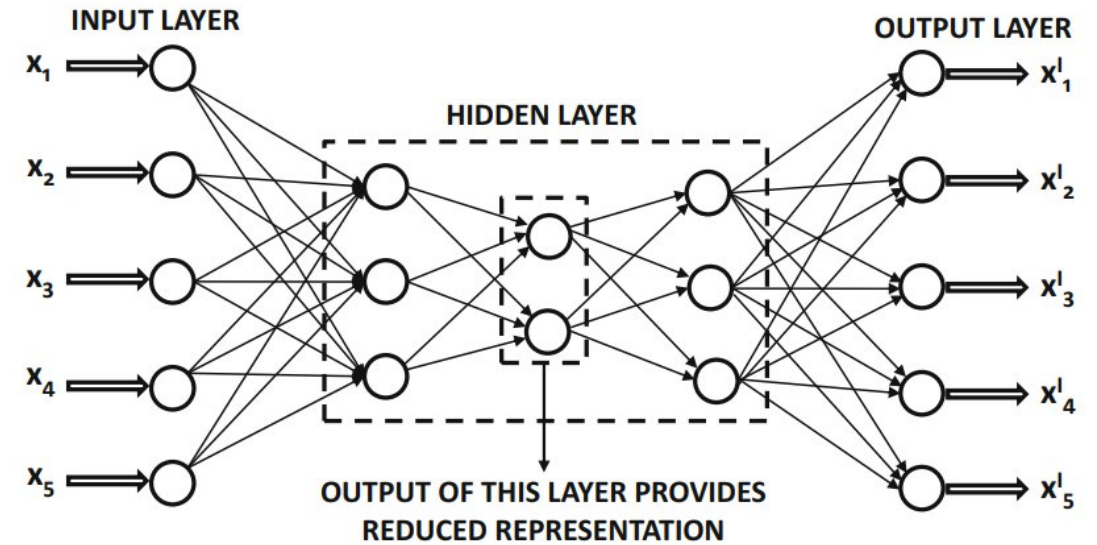
- Varianz in den Daten schlecht durch linearen Zusammenhang erklärbar?
—> Nichtlineare Aktivierungsfunktion, mehr Layer...

9.3 Vortrainieren mit Autoencodern

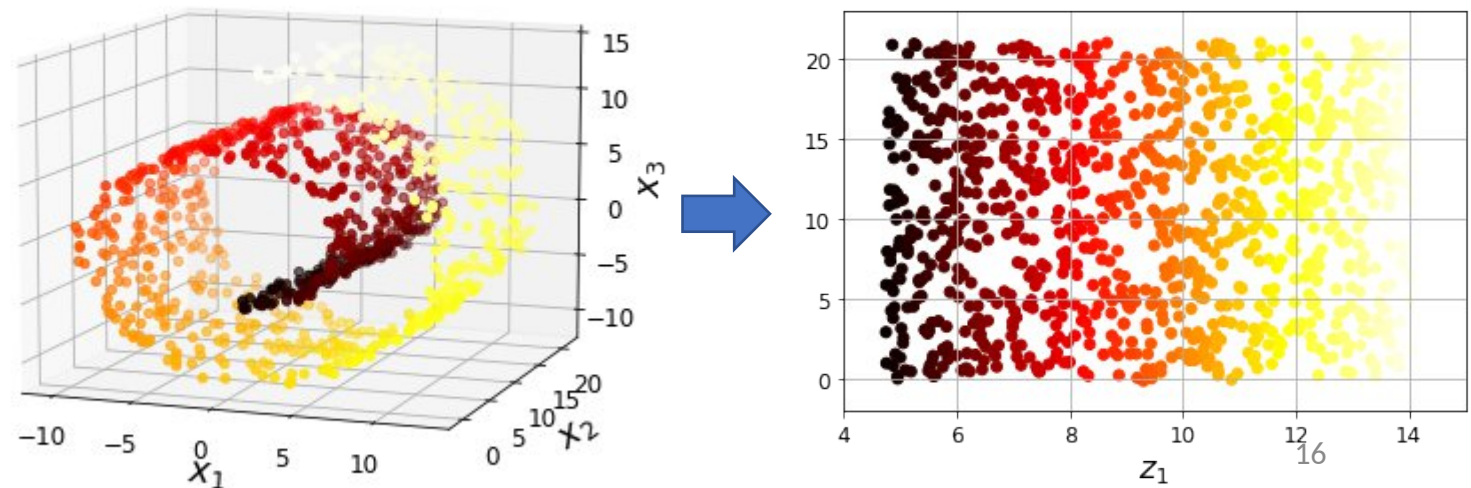
Deep Autoencoder

Autoencoder mit zusätzlichen Schichten und nichtlinearen Aktivierungsfunktionen können zur *nichtlinearen* Dimensionsreduktion eingesetzt werden

Diese können z.B. niedrigdimensionale Darstellungen von komplexen Klassenverteilungen lernen, auf denen die Klassen dann gut linear separiert werden können (—>Feature-Extraktion!)



Beispiel: Autoencoder mit 3 verdeckten Schichten

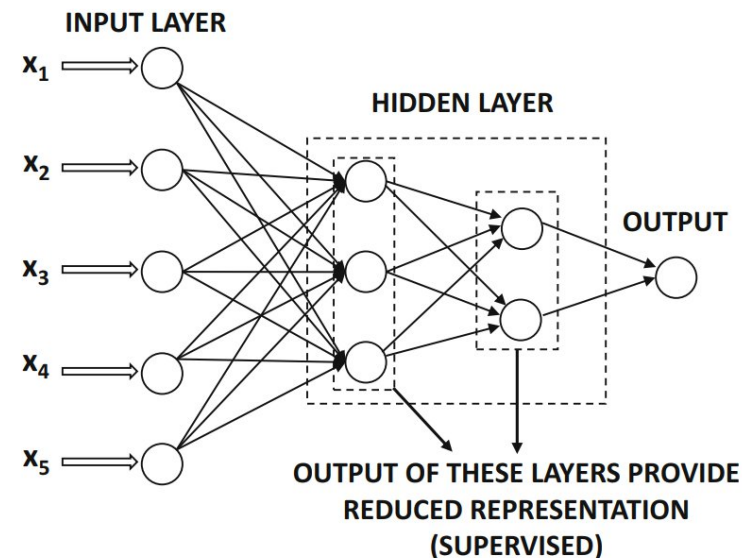


Anwendung: Vortrainieren

Wichtige Anwendung neben Dimensionsreduktion:

„Vortrainieren“ tiefer neuronaler Netze für überwachtes Lernen (z.B. zur Klassifizierung), d.h. gute Initialisierungen für die Gewichte finden —> kann Training enorm erleichtern.

Beispiel: Vortraining für ein neuronales Netz mit zwei verdeckten Schichten, das zur Klassifizierung eingesetzt werden soll.

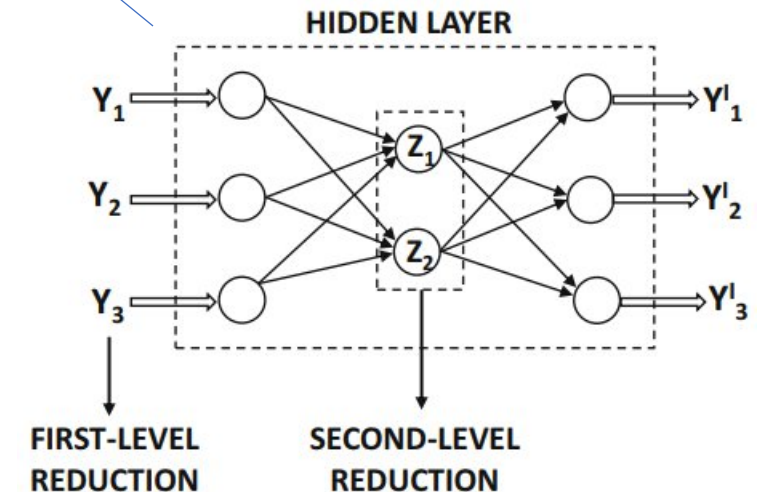
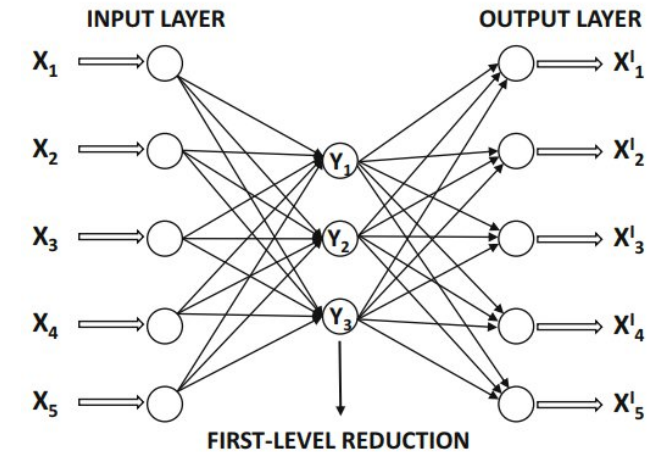
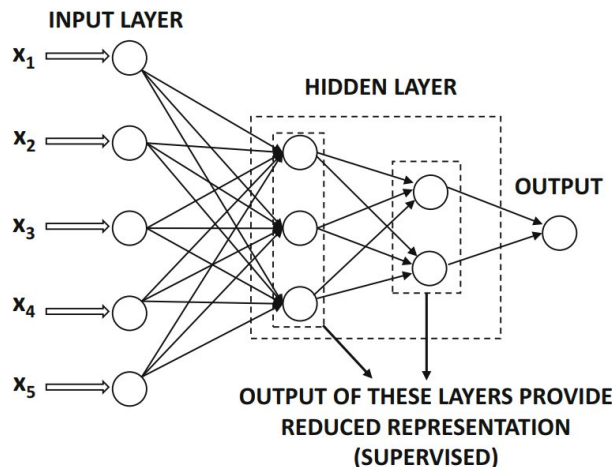


Anwendung: Vortrainieren

Schritt 1: Trainiere flachen Autoencoder auf Trainingsdaten um Initialisierung für die Gewichte der ersten Schicht zu erhalten.

Schritt 2: Trainiere flachen Autoencoder auf reduzierten Darstellung aus Schritt 1 um Initialisierung für die Gewichte der zweiten Schicht zu erhalten.

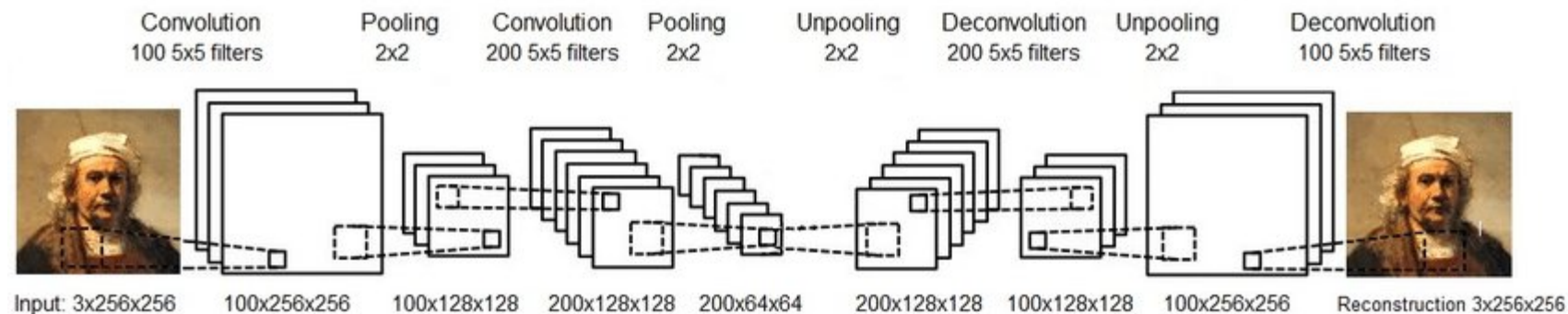
Schritt 3: Trainiere Klassifizierer mit initialen Gewichten aus Schritt 1 und 2. Normales Training, um Gewichte der Ausgabeschicht zu lernen und andere Gewichte anzupassen (fine-tuning).



9.4 Convolutional Autoencoder

Convolutional Autoencoder

Spezialfall, wenn im Encoder Faltungsschichten benutzt werden:

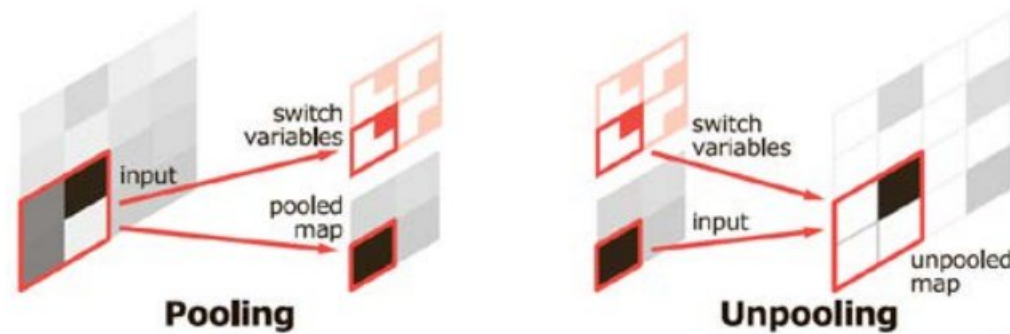


Neu: **Unpooling** und **Deconvolution (a.k.a. Transposed Convolution)**, die Pooling und Convolution umkehren.

Manchmal befindet sich in der Mitte noch ein fully connected oder flatten layer, so dass man eine vektorielle Darstellung hat (und nicht viele Featuremaps).

Unpooling

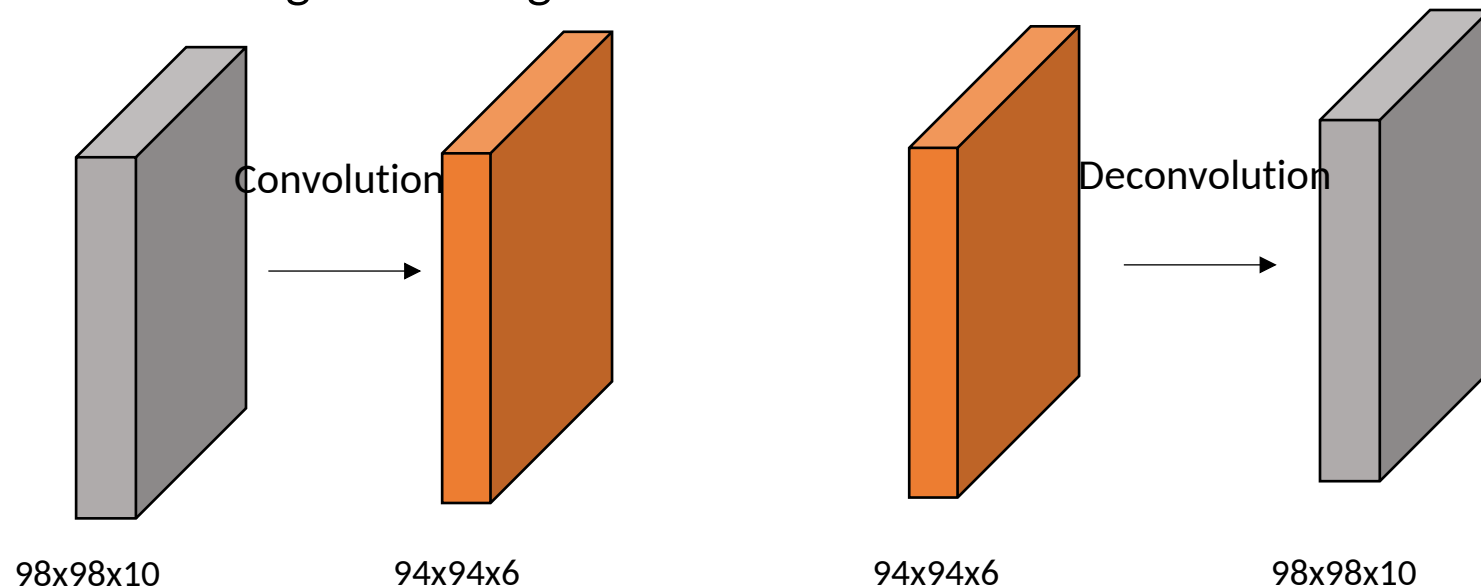
- Pooling = Downsampling, Unpooling = Upsampling
- Speichere die Position des maximalen Eintrages beim Pooling (switch variables).
- Unpooling setzt den maximalen Wert wieder an die richtige Stelle und füllt mit Nullen auf (—>Informationsverlust)



Deconvolution / transponierte Faltung

Deconvolution / transposed convolution ist nicht die mathematische Inverse der Faltung.

Idee: Die Transponierte Faltung ist eine Faltung, die man so aufbaut, dass sie Eingabe-/Ausgabedimension der zugeh. Faltung vertauscht.



Die Gewichte der Kernel der transponierten Faltung werden dabei auch gelernt, so dass sie potentiell die Umkehrung der Faltung lernen könnten (geht nicht immer)

Transponierte Faltung: Arithmetik

Dies wird durch Padding (kein/full/half) und Einfügen von Nullen zwischen den Eingabewerten erreicht.

Details: Vincent Dumoulin, Francesco Visin: “A guide to convolution arithmetic for deep learning”, <https://arxiv.org/abs/1603.07285>

Bsp.: Transponierte
Conv, wenn Stride = 2

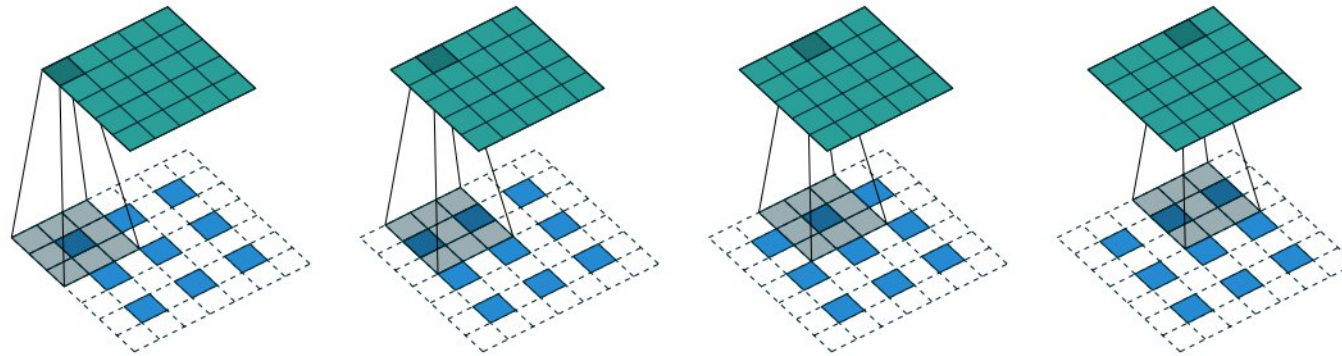


Figure 4.6: The transpose of convolving a 3×3 kernel over a 5×5 input padded with a 1×1 border of zeros using 2×2 strides (i.e., $i = 5$, $k = 3$, $s = 2$ and $p = 1$). It is equivalent to convolving a 3×3 kernel over a 3×3 input (with 1 zero inserted between inputs) padded with a 1×1 border of zeros using unit strides (i.e., $i' = 3$, $\tilde{i}' = 5$, $k' = k$, $s' = 1$ and $p' = 1$).

9.5 Anwendung: Anomalie- Erkennung mit Autoencodern

Anwendung: Anomalie-Erkennung in industriellen Prozessen

- Anomalien in Online-Prozessdaten können auf Ausfälle hinweisen, die zu gefährlichen Situationen oder Anlagenstillständen führen können
- Kernbestandteil des Alarmmanagements
- Üblich: händisch konfigurierte Alarmerkennung von Anomalien (z. B. Schwellwerte)
 - Schwierig kompliziertere Muster zu konfigurieren
 - Viele Fehlalarme



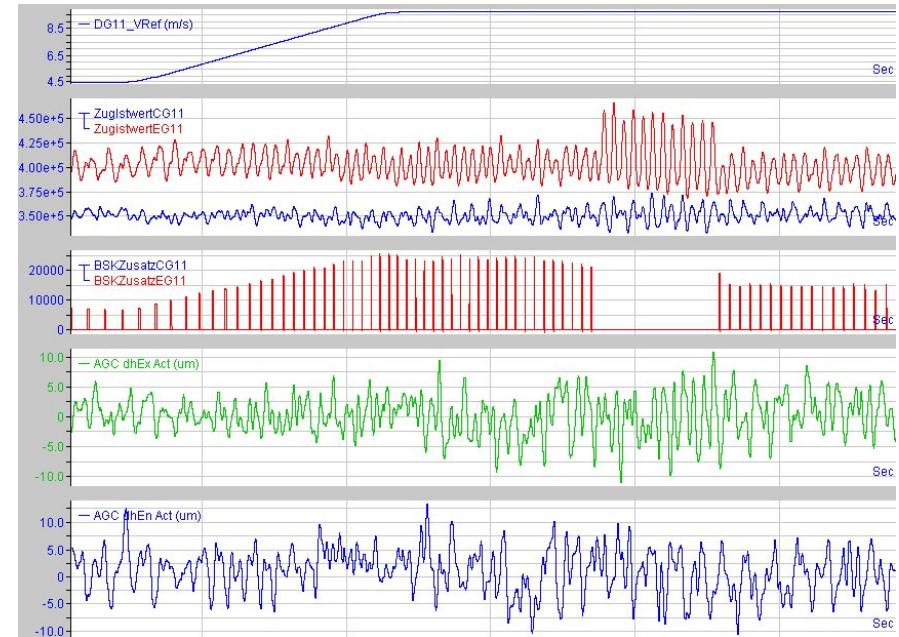
Leitstand

Ack/Prio	State	ActiveTime	ObjectName	ObjectDescription	Condition	Message
2	RTN	18 08:01:33:578	TL_2029_C29	COMP_C29_Temp_State	Temp-HH	Greater than 80.00
3	RTN	18 08:01:20:578	TL_2029_C29	COMP_C29_Temp_State	Temp-H	Greater than 75.00
2	RTN	18 08:00:52:078	TL_2077_A1	ReactorTemp_State	Temp-HH	Greater than 80.00
1	RTN	18 08:00:50:578	PL_2029_C29	COMP_C29_Pres_State	Pressure-LLL	Less than 5.00
2	RTN	18 08:00:48:077	PL_2029_C29	COMP_C29_Pres_State	Pressure-LL	Less than 10.00
3	ACT	18 08:00:42:578	PL_2029_C29	COMP_C29_Pres_State	Pressure-L	Less than 20.00
3	ACT	18 08:00:36:078	TL_2077_A1	ReactorTemp_State	Temp-H	Greater than 75.00
1	RTN	18 07:59:27:577	TL_2077_A1	ReactorPressure_State	Pressure-LLL	Less than 5.00
2	RTN	18 07:59:24:577	PL_2077_A1	ReactorPressure_State	Pressure-LL	Less than 10.00
3	RTN	18 07:59:18:078	PL_2077_A1	ReactorPressure_State	Pressure-L	Less than 20.00
1	RTN	18 07:58:53:078	TL_2029_C29	COMP_C29_Temp_State	Temp-LLL	Less than 5.00
2	RTN	18 07:58:48:578	TL_2029_C29	COMP_C29_Temp_State	Temp-LL	Less than 10.00
3	RTN	18 07:58:40:077	TL_2029_C29	COMP_C29_Temp_State	Temp-L	Less than 20.00
1	RTN	18 07:55:58:578	TL_2077_A1	ReactorTemp_State	Temp-LLL	Less than 5.00
2	RTN	18 07:55:49:078	TL_2077_A1	ReactorTemp_State	Temp-LL	Less than 10.00
3	RTN	18 07:55:32:078	TL_2077_A1	ReactorTemp_State	Temp-L	Less than 20.00
1	ACT	15 09:51:43:578	Compressor-Drive	Compressor-Drive	DischargeLow	Compressor-DriveTorgue-Low
1	ACT	15 09:51:43:578	Reactor_Bypass	Reactor_Bypass	Red	Reactor_ByPassByPass_Blk
1	ACT	15 09:51:43:578	Compressor-Hydraulic	Compressor-Hydraulic	Cavitating	Compressor-HydraulicOil-Press-Low
1	ACT	15 09:51:43:578	IsolationSystem	IsolationSystem	Unstable	IsolationSystemHelium_Gas_ON

Liste von Alarmen

Prozessindustrie: Typische Situation für kontinuierliche Prozesse

1. Viele gespeicherte Daten, aber oft redundant und wenig Variation
2. Wenige Anomalien
3. Für Labeling von Anomalien wird Fachwissen benötigt —> nur wenige Menschen kommen dafür in Frage



Idee: Rekonstruktionsbasierte Anomalie-Erkennung

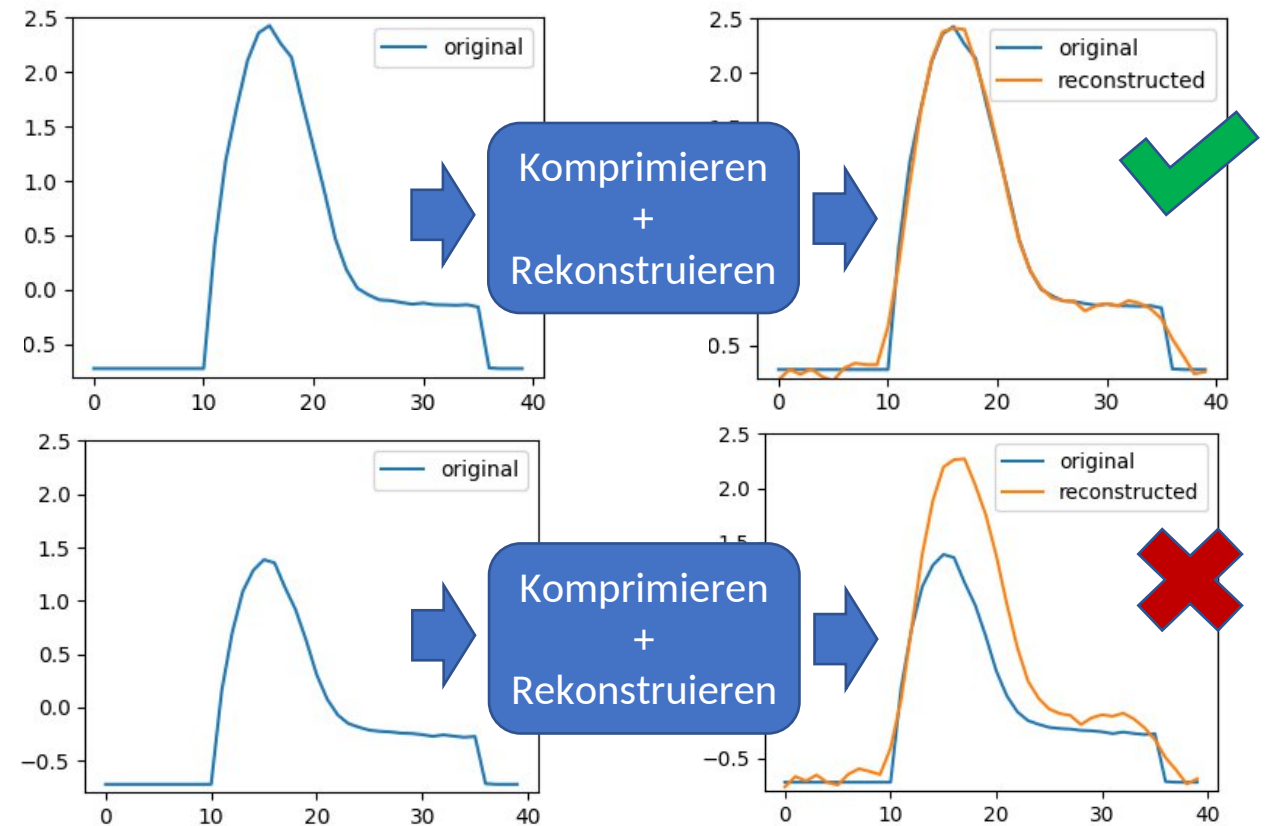
Idee

1. Lerne verlustbehaftete Komprimierung von normalen Daten durch Autoencoder inkl. Statistik über typischen Rekonstruktionsfehler.
2. Ungesehene Daten werden ebenfalls komprimiert und rekonstruiert. Falls der Rekonstruktionsfehler größer ist als im Training, liegt eine Anomalie vor.

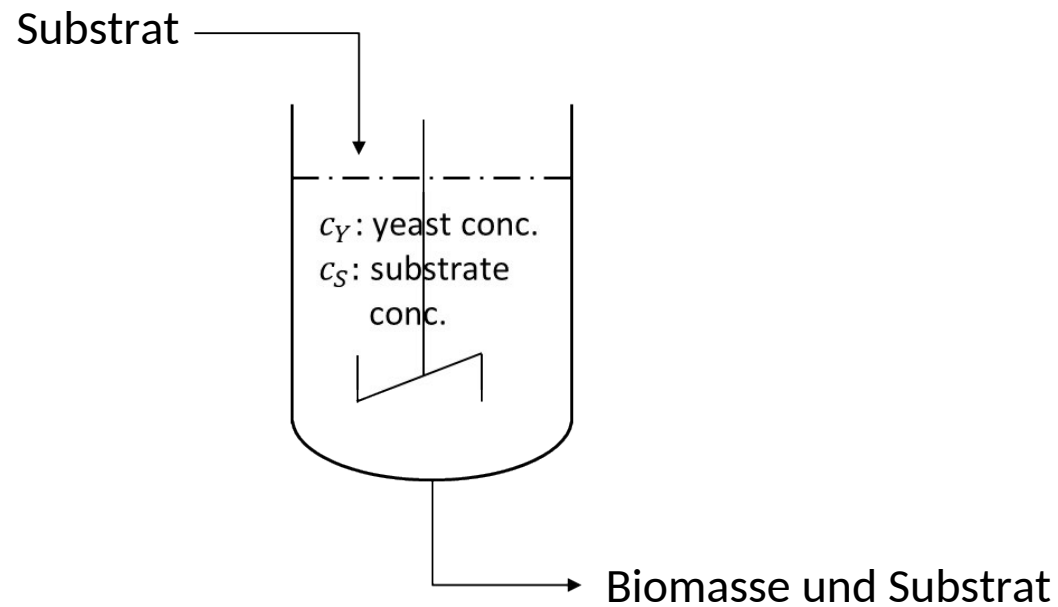
Benutzung im laufenden Betrieb

Komprimiere und rekonstruiere auf *rollierendem Fenster*. Überwache dabei den Rekonstruktionsfehler.

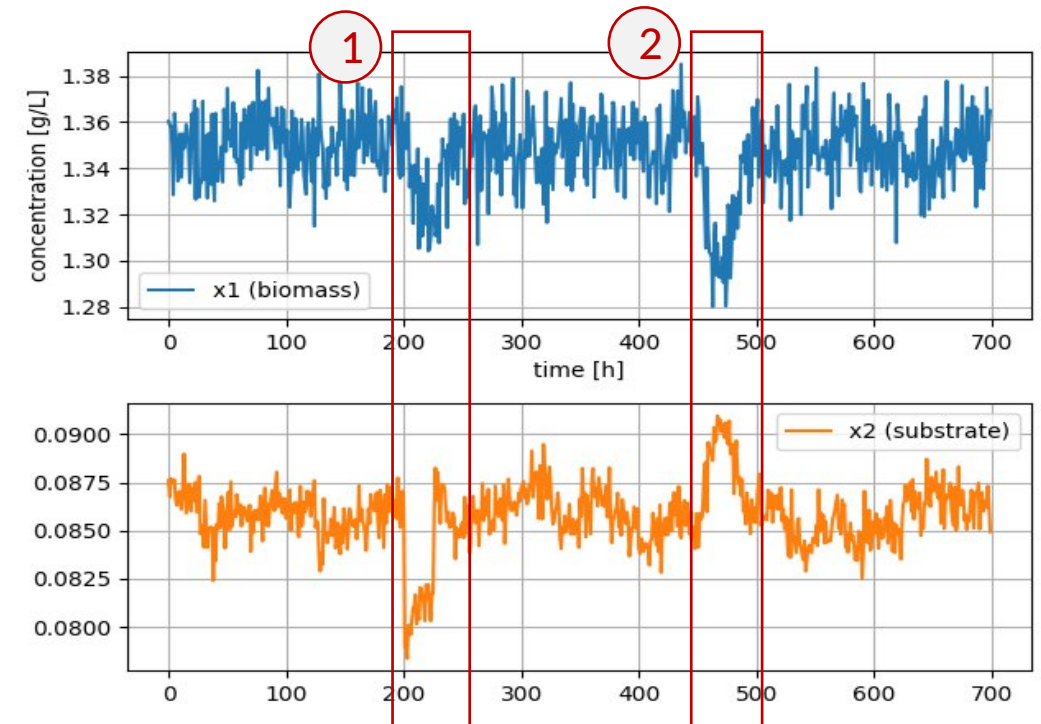
Alarm, falls Fehler > Schwellwert.



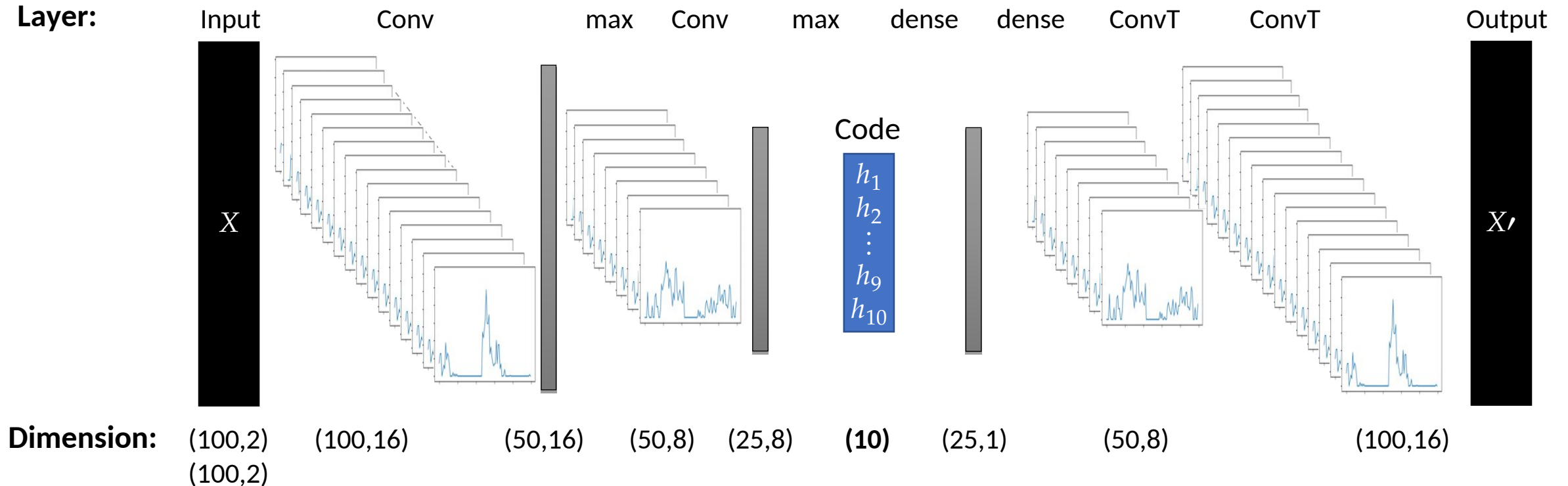
Beispiel: Hefeproduktion



- Kontinuierlicher Prozess: Hefeproduktion
- Daten: Zeitreihen für Konzentration von Substrat und Biomasse
- Anomalien: z.B. verstopfte Zu-/Abläufe (1), erhöhte Hefezellensterberate (2), ...



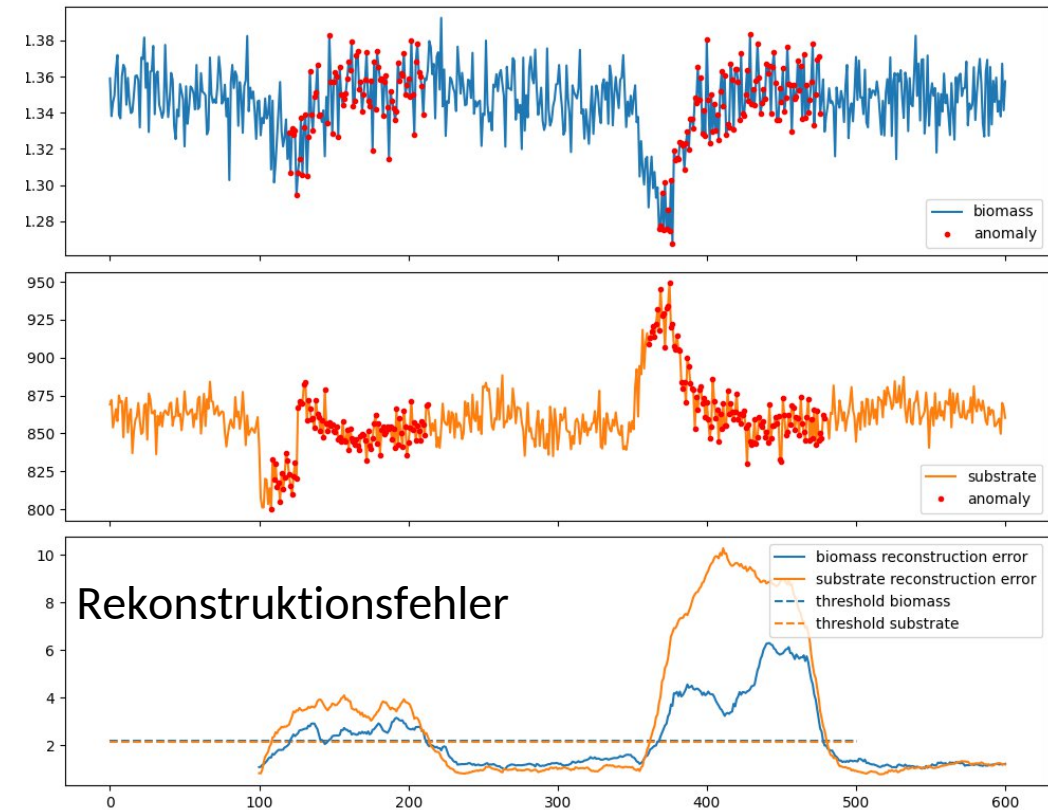
Beispielarchitektur: Convolutional Autoencoder



- Trainingsdaten: 9250 Instanzen (2-Dim-Zeitreihen mit 100 Samples, standardisiert)
Trainiert mit SGD für 5 Epochen (Batchgröße 128) (Verwendete Software: **K** Keras)

Beispiel: Ergebnisse

Roter Punkt: Fenster mit diesem Endpunkt (200 Samples) wurde als Anomalie erkannt
(Rekonstruktionsfehler zu hoch, siehe unterer Graph)



Zusammenfassung Autoencoder

- Spezielle Architekturtyp, bei dem eine niedrig-dimensionale Repräsentation (latent space) der Trainingsdaten gelernt wird.
- Netz wird darauf trainiert, den Input vorherzusagen. Latent space kann per post-processing aus dem trainierten Netz extrahiert werden.
- Enger Zusammenhang zu spektralen Methoden (PCA, SVD, Matrixfaktorisierung), kann diese verallgemeinern.
- Flexibler Aufbau der Schichten (Beispiele: convolutional, fully connected)
- Wird u.a. verwendet, um nachgelagertes Training für supervised learning Problem zu erleichtern (Vortrainieren einzelner Schichten).
- Kann zur Erkennung von Anomalien (unsupervised) eingesetzt werden.

9.6 Zusatz: Matrixfaktorisierung als neuronales Netz

Anwendung: Recommender Engines

Filmempfehlungen:

Rating r_{ij} von user i
für item (hier: Film) j
z.B. mit Werten
zwischen 0 und 5

Können wir User3 den
Film Inception
empfehlen?



	Item1	Item2	Item3
User1	5	4	0
User2	5	5	1
User3	0	???	5
User4	0	0	4

—> Siehe Vorlesungsteil Prof. Preisach!

Mathematisch: Eine Rating-Matrix R soll faktorisiert werden, so dass $R \approx PQ^T$ mit Rank- k Matrizen P und Q (k : gewählte Anzahl an erklärenden Faktoren).

Wir würden eigentlich gern die SVD nehmen (würde automatisch die k besten Faktoren auswählen!)
aber: R ist gar nicht vollständig bekannt, da nicht jeder User jeden Film bewertet hat.

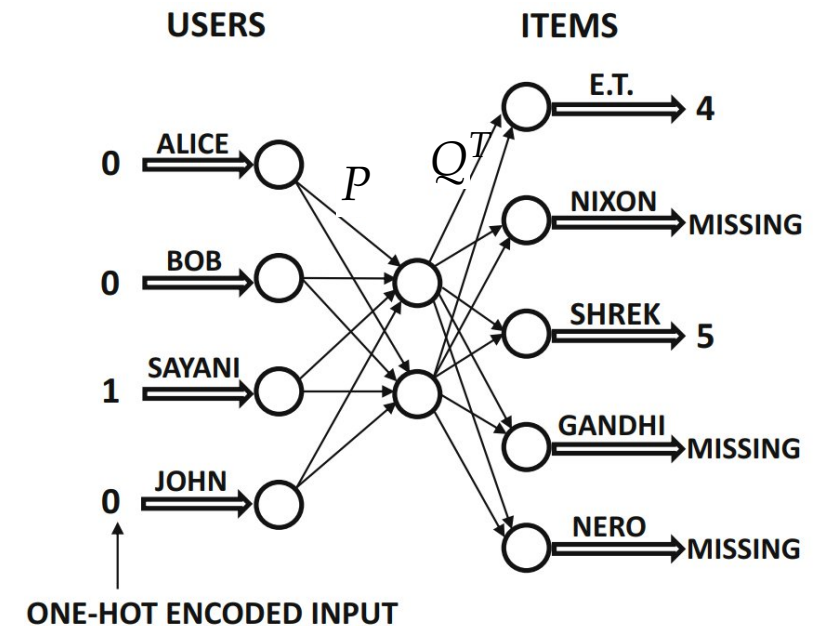
Autoencoder für Recommender Engines

Architektur:

- Eingabeschicht: Anzahl Nutzer. Vorhersage für einen bestimmten Nutzer mittels One-Hot-Encoded Vektor.
- Verdeckte Schicht: Anzahl Faktoren k
- Ausgabeschicht: Anzahl Filme

Für einen Nutzer fehlen Bewertungen, d.h. wir können keine Verlustfunktion für diese Outputs berechnen, aber wir können für *alle* Filme Vorhersagen machen.

Wie trainiert man?



Autoencoder für Recommender Engines

Wie trainiert man?

Für jeden Nutzer wird der Verlust nur für die Output-Knoten berechnet, für die Werte beobachtet wurden.

(Interpretation: jeder Datenpunkt (=User) wird auf einem etwas anderen NN trainiert, aber mit denselben Gewichten)

Das Ganze kann man auch elementweise machen, also für alle Paare (i, j) für die eine Bewertung r_{ij} bekannt ist. In diesem Fall hat man als Trainingsmenge Netze mit jeweils einem Output.

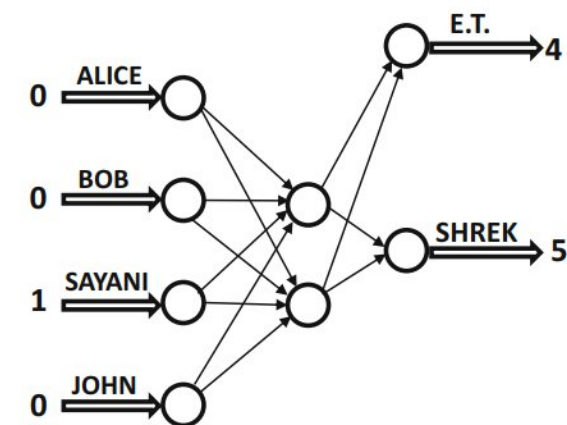
Der Vorhersage für ein Beispiel (User i , Film j) ist dann:

$$e_i P Q^T = p_i^T q_j \approx r_{ij}$$

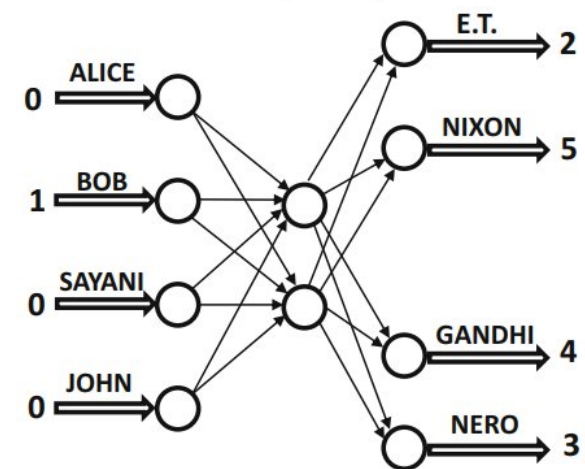
One-hot encoded
Zeilenvektor

i -te Zeile von P , j -te
Zeile von Q

Damit ist die Verlustfunktion identisch wie bei der unvollständigen Matrixfaktorisierung.



OBSERVED RATINGS (SAYANI): E.T., SHREK



OBSERVED RATINGS (BOB): E.T., NIXON, GANDHI, NERO